Mini Paper — 2.2 Problem Solving & Programming

Time: ~35 minutes · Total: 30 marks · Answer all questions.

Mark your work against the mark scheme at the end; aim for ~1 minute per mark.

Questions

Q1. State the difference between a **function** and a **procedure**. **[2]** (AO1)

Q2. Explain what is meant by **recursion**, and describe one problem that can occur if a recursive subroutine is written incorrectly. **[4]** (AO1)

Q3. The recursive function below is called as `total(4)`.

```
function total(n)
    if n == 0 then
        return 0
    else
        return n + total(n - 1)
    endif
endfunction
```

(a) Using the call stack, trace the calls made and show the values returned as the stack unwinds. **[3]** (AO2)

(b) State the **final value** returned by `total(4)`. **[1]** (AO2)

Q4. A programmer uses the procedures below.

```
procedure changeByValue(byVal x)
    x = x + 10
endprocedure

procedure changeByReference(byRef x)
    x = x + 10
endprocedure
```

The following code runs:

```
num = 5
changeByValue(num)
print(num)
changeByReference(num)
print(num)
```

(a) State the two values that are printed, in order. **[2]** (AO2)

(b) Explain **why** the two outputs differ, referring to how the parameter is passed in each case. **[3]** (AO1)

Q5. A program to manage a library stores information about books.

Write, in **OCR pseudocode**, a class `Book` that has:

- two **private** attributes: `title` and `copies`
- a **constructor** that sets both attributes from parameters
- a procedure `borrow()` that reduces `copies` by 1 only if `copies` is greater than 0
- a **getter** `getCopies()` that returns the number of copies. **[6]** (AO3)

Q6. Define the term **encapsulation** and give **one** reason it is used in object-oriented programming. **[3]** (AO1)

Q7. A streaming service wants to analyse millions of past viewing records to discover which genres of film are often watched together, so it can recommend titles.

(a) Name the **computational method** best suited to this task. **[1]** (AO1)

(b) Justify your choice with reference to the scenario. **[2]** (AO2)

Q8. Explain the difference between a **local variable** and a **global variable**, and give **one** reason why a programmer would prefer to use local variables. **[3]** (AO1)

Mark scheme

Q1 — [2] (AO1) - A function returns a (single) value (1). - A procedure performs a task but does not return a value (1).

Examiner tip: State the defining difference (returns a value vs does not) — do not write that a function is "more complex".

Q2 — [4] (AO1) - Recursion is a subroutine that calls itself (1). - A correct recursive routine must have a base case that stops the recursion (1). - If the base case is missing or never reached, the routine calls itself indefinitely (1). - Each call adds a frame to the call stack until memory is exhausted — a stack overflow (1). *(Max 4)*

Examiner tip: Always pair "calls itself" with "base case"; name **stack overflow** for the error.

Q3 — [4] (AO2)

(a) **[3]** - Calls pushed in order: `total(4)` → `total(3)` → `total(2)` → `total(1)` → `total(0)` (base case) (1 for correct order of pushes). - Base case `total(0)` returns 0 (1). - Unwinding (popping): total(1)=1+0=1; total(2)=2+1=3; total(3)=3+3=6; total(4)=4+6=10 (1 for correct unwinding).

(b) **[1]** - Final value returned = **10** (1).

Examiner tip: Show frames being pushed then popped in LIFO order — do not just compute the answer.

Q4 — [5]

(a) **[2] (AO2)** - First output: **5** (1). - Second output: **15** (1).

(b) **[3] (AO1)** - `changeByValue` passes a copy of the value (1), so the original `num` is unchanged and 5 is printed (1). - `changeByReference` passes the memory address of `num` (1), so the procedure alters the original and 15 is printed (1). *(Max 3)*

Examiner tip: Examiners want the **consequence** — original unchanged vs original changed — not just "a copy is made".

Q5 — [6] (AO3)

Award up to 6, e.g.: - `class Book` declared with two **private** attributes (1). - Constructor `new` taking parameters and assigning both attributes (1). - `borrow()` reduces `copies` by 1 (1) only when `copies > 0` (selection) (1). - Getter `getCopies()` that **returns** copies (1). - Correct OCR syntax / encapsulation (private attributes, public methods) (1).

```
class Book
    private title
    private copies

    public procedure new(givenTitle, givenCopies)
        title = givenTitle
        copies = givenCopies
    endprocedure

    public procedure borrow()
        if copies > 0 then
            copies = copies - 1
        endif
    endprocedure

    public function getCopies()
        return copies
    endfunction
endclass
```

Examiner tip: Mark `private` on the attributes and a `return` in the getter — both are easy marks candidates often miss.

Q6 — [3] (AO1) - Encapsulation is bundling attributes (data) and the methods that act on them inside a class (1). - Attributes are kept private and accessed only through public methods (1). - Reason: protects data integrity / data hiding / prevents invalid or accidental changes / allows validation in setters (1).

Examiner tip: Keep encapsulation (private data + public methods) distinct from inheritance and polymorphism.

Q7 — [3]

(a) **[1] (AO1)** - **Data mining** (1).

(b) **[2] (AO2)** - It searches a very large data set (millions of records) to find patterns/correlations not obvious before (1). - Here it reveals which genres are watched together, which informs recommendations (1).

Examiner tip: Always link the method to the scenario detail (large data set, finding hidden patterns) to earn the justification marks.

Q8 — [3] (AO1) - A local variable is declared inside a subroutine and is accessible only within it (1). - A global variable is declared in the main program and is accessible anywhere in the program (1). - Reason to prefer local: fewer side effects / avoids accidental changes from elsewhere / frees memory when the subroutine ends / more modular and easier to debug (1).

Examiner tip: Scope is about **visibility**, not whether the value can change — do not confuse a global with a constant.

Total: 30 marks (Q1 2 + Q2 4 + Q3 4 + Q4 5 + Q5 6 + Q6 3 + Q7 3 + Q8 3).